

GlideQuery: chain rules, Stream, and Optional

This note explains why a GlideQuery chain stops, how to store a query and add to it, and which methods belong to Stream versus Optional. Names follow the official API: **orderBy** / **orderByDesc**, and **deleteMultiple**. **orderByAsc** and a GlideQuery **delete()** are not in the documented method list. In a scoped app the class is **global.GlideQuery**.

1. The three objects

GlideQuery is not one object with every method on it. A script holds one of three things, and a method exists only on the object you are holding.

Object	How you get it	What you can call
GlideQuery	new GlideQuery(table), or any intermediate call	where, orWhere, whereNull, whereNotNull, orWhereNull, orWhereNotNull, orderBy, orderByDesc, limit, groupBy, aggregate, having, withAcls, disableWorkflow, forceUpdate, disableAutoSysFields
Stream	select()	map, filter, limit, flatMap (still a Stream). forEach, reduce, toArray, find (these end the Stream). find returns an Optional.
Optional	selectOne(), get(), getBy(), insert, update, insertOrUpdate, Stream.find()	isPresent, isEmpty, ifPresent, map (still an Optional). get and orElse unwrap it.

Intermediate GlideQuery methods return a new GlideQuery, so they chain. A terminal GlideQuery method leaves the builder. select returns a Stream. selectOne, get, getBy, insert, and update return an Optional. count returns a number. updateMultiple returns an object with rowCount. deleteMultiple returns nothing. parse is static: GlideQuery.parse(table, encodedQuery). It is not a method on an instance.

2. Basics

Compared with GlideRecord, you do not call query() or next(). Each builder call returns a new object instead of mutating one record pointer. Field values come back as plain JavaScript values, so a string comparison with === works. A typo in a field name throws NiceError and runs nothing. The same typo on GlideRecord is ignored, and a following deleteMultiple can match every row.

What GlideQuery is — and what it is not

A server-side fluent API for reading and writing records. You build a query, then one terminal call runs it. In a scoped app the class is **global.GlideQuery**.

COMPARED WITH GLIDERECORD

- GlideRecord mutates one object: **addQuery()**, then **query()**, then **next()**.
- GlideQuery returns a new object from each builder call. You do not call **query()** or **next()**.
- A bad field name fails fast. GlideRecord often ignores it and matches every row.
- Results are plain objects, not GlideElement. **===** works on strings.

THREE OBJECTS, NOT ONE

- **GlideQuery** — the builder. `where` `orderBy` `limit`
- **Stream** — many rows, from **select()**. `forEach` `map`
`toArray`
- **Optional** — one row or none, from **selectOne()** / **get()**.
`orElse` `isPresent`

A method exists only on the object you are holding. That is the whole chaining rule.

THE SHAPE OF A READ

```
new GlideQuery('incident')
  .where('active', true)
  .orderBy('number')
  .limit(20)
  .select('number')
  .forEach(function (row) {
    gs.info(row.number);
  });
```

Builders first. **select** hands you a Stream. **forEach** is what actually pulls rows.

STORE THE BUILDER, THEN ADD TO IT

```
var open = new GlideQuery('incident')
  .where('active', true);

var p1 = open.where('priority', 1).count();
var p2 = open.where('priority', 2).select('number');

// p2 is a Stream — no where()
p2.where('priority', 1);
```

Each **where** returns a new GlideQuery. **open** is not changed. **count()** returns a number. **select()** returns a Stream.

FAIL-FAST, BEFORE ANY WRITE

- Unknown field: **activated** instead of **active** throws. It does not match every row.
- Bad choice and wrong type throw the same way.
- **update()** needs a primary-key **where**, usually **sys_id**.
- Sets use **updateMultiple()**. Deletes use **deleteMultiple()**. There is no single-record delete.
- Run the same **where** chain through **count()** before a bulk write.

Figure 1. What GlideQuery is, how a stored builder is extended, and the fail-fast write rules.

A normal read

```
new GlideQuery('incident')
  .where('active', true)
  .orderBy('number')
  .limit(20)
  .select('number', 'short_description')
  .forEach(function (row) {
    gs.info(row.number + ' ' + row.short_description);
  });
```

where, orderBy, and limit are still a GlideQuery. select is the handoff: it returns a Stream, and sys_id is always included even if you did not ask for it. forEach is a Stream method. That is the call that pulls rows. select itself is lazy — mapping and filtering on the Stream do no work until forEach, reduce, toArray, or find.

3. Why select().where() fails

where is a method on GlideQuery. select does not return a GlideQuery, so the next call is not looking up where on the builder. It is looking up where on a Stream, and Stream has no where. The failure is a missing method, not a rejected filter.

```
new GlideQuery('incident') // GlideQuery
  .where('active', true) // GlideQuery
  .select('number') // Stream — builder is over
```

```
.where('priority', 1); // Stream has no where()
```

Terminal means the GlideQuery chain is over. It does not mean the database already ran. You can still map and filter the Stream. You cannot go back and add a query builder method. If the filter belongs in the query, put where before select. Stream.filter after select filters rows already headed for script, which is a different operation.

4. Storing a query and adding to it

You store the builder. Intermediate calls return a new GlideQuery, so a variable can be extended. The graphic that says a terminal returns nothing GlideQuery-shaped is only about that last call. Calls before it are how the variable stays usable.

```
var open = new GlideQuery('incident').where('active', true);

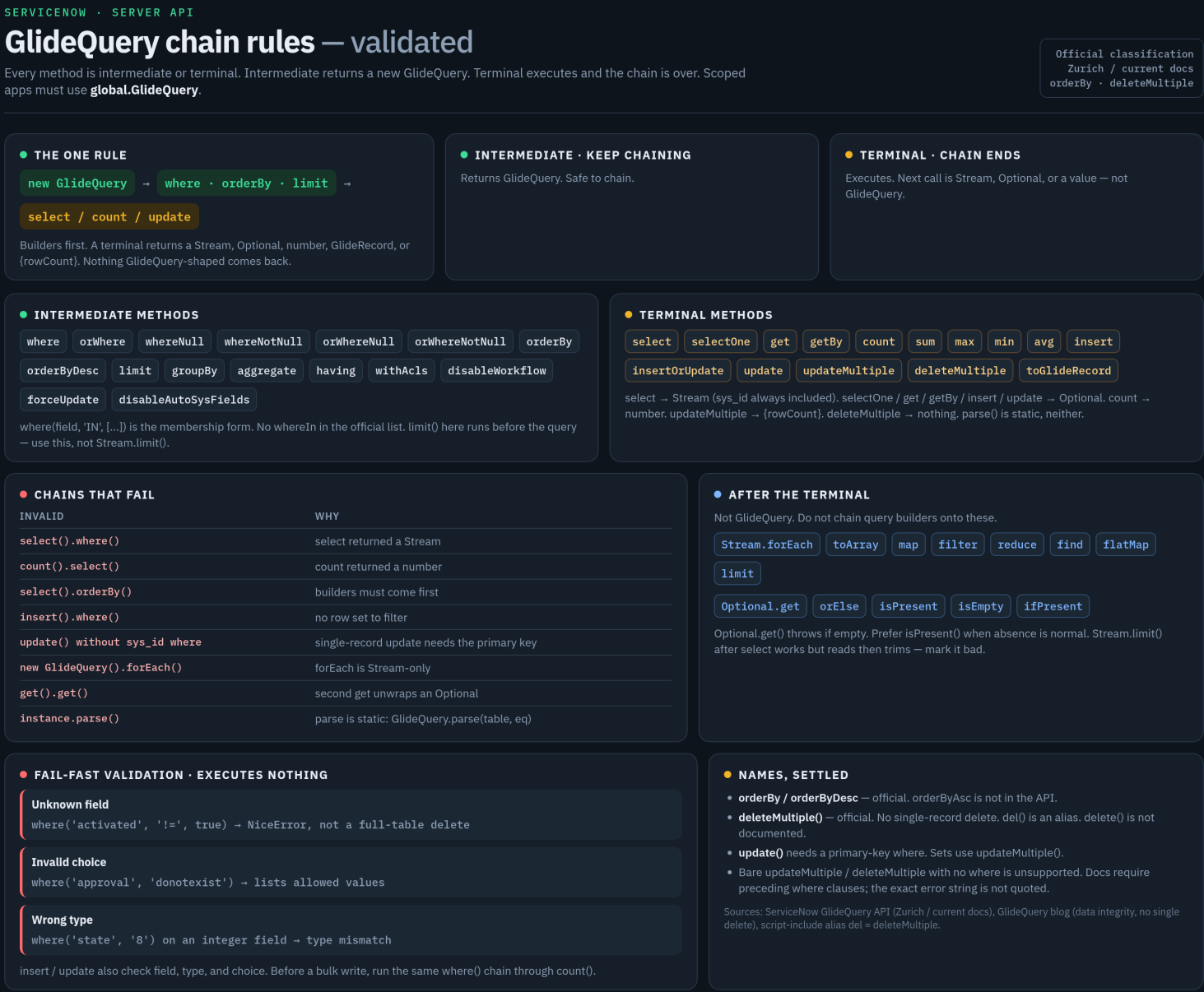
// each where returns a new GlideQuery; open is unchanged
var p1 = open.where('priority', 1).count(); // number
var p2 = open.where('priority', 2).select('number'); // Stream

p2.where('priority', 1); // invalid: p2 is a Stream
```

Reassigning also works, because the right-hand side is still a GlideQuery: incidents = incidents.where('priority', 1).orderBy('number'). The terminal stays last. count and select are not stored if you still need to add conditions.

5. Chain rules

Green methods return a GlideQuery. Amber methods execute and end the builder. Blue methods belong to Stream or Optional. Red rows are calls that fail because the receiver is the wrong object, or because the write is unbounded.



Invalid chains

Call	Why it fails
<code>select().where() / select().orderBy()</code>	<code>select</code> returned a <code>Stream</code> . Builders must come first.
<code>count().select()</code>	<code>count</code> returned a number.
<code>insert().where()</code>	An insert has no row set to filter.
<code>update()</code> without a <code>primary-key</code> where	Single-record update requires the primary key, usually <code>sys_id</code> . Use <code>updateMultiple</code> for a set.
<code>new GlideQuery(table).forEach(fn)</code>	<code>forEach</code> is <code>Stream</code> -only. <code>select</code> first.
<code>get().get()</code>	The first <code>get</code> is <code>GlideQuery.get(sys_id)</code> . The second is <code>Optional.get</code> , and it throws if empty.
<code>instance.parse(...)</code>	<code>parse</code> is static: <code>GlideQuery.parse(table, encodedQuery)</code> .
<code>selectOne().forEach(fn)</code>	<code>selectOne</code> returned an <code>Optional</code> . <code>Optional</code> has no <code>forEach</code> .
<code>orElse(v).isPresent()</code>	<code>orElse</code> already unwrapped the <code>Optional</code> . The result is a plain value.

6. Stream versus Optional

Use `select` when you expect many rows, including zero. Use `selectOne`, `get`, or `getBy` when you expect one row that might be missing. `get(sys_id)` looks up the primary key. `getBy({field: value})` returns one match even if several rows qualify. `insert`, `update`, and `insertOrUpdate` also return an `Optional` of the written row.

SERVICENOW · AFTER THE TERMINAL

Stream vs Optional — valid and invalid

`select()` returns a `Stream` (many rows). `selectOne()`, `get()`, and `getBy()` return an `Optional` (one row, or none). Neither has `where()`.

STREAM — FROM SELECT()

Still a `Stream`, so you can chain them: `map` `filter` `limit` `flatMap`

These end the `Stream`: `forEach` `reduce` `toArray` `find` — `find` returns an `Optional`.

```
new GlideQuery('incident')
  .where('active', true)
  .select('number')
  .filter(function (row) { return row.number; })
  .map(function (row) { return row.number; })
  .forEach(gs.info);
```

```
var rows = new GlideQuery('incident')
  .select('number').toArray(50);
```

```
.select('number').where('active', true);
.select('number').orderBy('number');
.forEach(gs.info).map(fn);
new GlideQuery('incident').filter(fn);
```

`map/filter/limit` are lazy. Rows are read when `forEach`, `reduce`, `toArray`, or `find` runs. `Stream.limit` trims after the query. `GlideQuery.limit`, before `select`, is the one that belongs in the query.

OPTIONAL — FROM SELECTONE / GET / GETBY

`isPresent` `isEmpty` return a boolean. `get` returns the row or throws. `orElse` returns the row or a default. `ifPresent` runs only if a row exists. `map` returns another `Optional`.

```
var user = new GlideQuery('sys_user')
  .where('email', 'abel.tuter@example.com')
  .selectOne('first_name')
  .orElse({ first_name: 'Nobody' });

var opt = new GlideQuery('sys_user')
  .selectOne('first_name');
if (opt.isPresent()) gs.info(opt.get().first_name);

.selectOne('first_name').where('active', true);
.selectOne('first_name').forEach(gs.info);
.selectOne('first_name').get(); // throws if empty
.orElse({ first_name: 'Nobody' }).isPresent();
```

`orElse` already unwrapped the `Optional`, so the result is a plain object. `insert`, `update`, and `insertOrUpdate` also return an `Optional`. `Stream.find` is the bridge: `Stream` in, `Optional` out.

Figure 3. Valid and invalid syntax once the builder has ended.

Stream

map, filter, limit, and flatMap return a Stream, so they chain, and they are lazy. forEach, reduce, and toArray end the Stream and run the query. find ends the Stream and returns an Optional, so Optional methods are legal after it. toArray(n) needs a cap. Prefer GlideQuery.limit before select. Stream.limit works, then trims, which reads more than you keep.

```
new GlideQuery('incident')
  .where('active', true)
  .select('number', 'short_description')
  .filter(function (row) { return row.short_description; })
  .map(function (row) { return row.number; })
  .forEach(gs.info);

var rows = new GlideQuery('incident').where('active', true).select('number').toArray(50);

new GlideQuery('task').select('description')
  .find(function (row) { return row.description; })
  .ifPresent(function (row) { gs.info(row.sys_id); });
```

Optional

isPresent and isEmpty return a boolean. get returns the row or throws "get() called on empty Optional" — use it only when a missing row is a bug. orElse(default) returns the row or the default, already unwrapped. ifPresent(fn) runs only when a row exists. map(fn) returns another Optional, so it can sit before orElse or get.

```
var user = new GlideQuery('sys_user')
  .where('email', 'abel.tuter@example.com')
  .selectOne('first_name', 'email')
  .orElse({ first_name: 'Nobody', email: '' });

var opt = new GlideQuery('sys_user').where('last_name', 'Luddy').selectOne('first_name');
if (opt.isPresent()) {
  gs.info(opt.get().first_name);
}
```

7. Writes

update needs a where on the primary key and returns an Optional of the updated row. updateMultiple takes a changes object and returns {rowCount}. There is no single-record delete. deleteMultiple deletes the rows matched by the preceding where clauses; del is an alias of deleteMultiple, not a second API. Docs describe both bulk writes as operating on the preceding where clauses. A bare updateMultiple or deleteMultiple with no where is unsupported. The exact error string is not quoted in the docs, so pair the same where chain with count before either call. Do not use deleteMultiple on tables with currency fields; currency child rows can be orphaned. Delete those one by one.

```
var n = new GlideQuery('incident').where('active', true).where('priority', 1).count();
gs.info('would update ' + n);

new GlideQuery('incident')
  .where('sys_id', '0123456789abcdef0123456789abcdef')
```

```

.update({ short_description: 'corrected' });

new GlideQuery('incident')
.where('active', false)
.where('priority', 5)
.updateMultiple({ urgency: 3 });

```

8. Names that cheat sheets disagree on

Use this	Not this	Note
orderBy / orderByDesc	orderByAsc	orderByAsc appears on some unofficial sheets. It is not in the official method list.
deleteMultiple	delete() / Del()	No single-record delete. del is the script-include alias of deleteMultiple.
where(field, 'IN', [...])	whereIn / whereNotIn	IN is an operator on where, not a separate documented method.
global.GlideQuery	GlideQuery in a scoped app	Scoped scripts must prefix the global scope.
GlideQuery.parse(table, eq)	query.parse(...)	parse is static.

Sources: ServiceNow GlideQuery API (Zurich developer reference and current scoped/global docs), the GlideQuery developer blog on data integrity and on there being no single-record delete, the Stream and Optional API pages, and the script-include alias of del to deleteMultiple. The method surface has changed across releases. If a name here is missing in a Background Script, inspect `GlideQuery.prototype` on that instance.